

Day 2

Docker Lab 3 Short Note

Docker Lab 3 focused on Docker images, Dockerfile instructions, image building, and container deployment.

In this lab, image management operations such as listing images, checking image size, and inspecting image details were performed. Dockerfile instructions including FROM, COPY, RUN, WORKDIR, EXPOSE, and ENTRYPOINT were studied and used to build a custom Docker image.

The lab also covered:

- Building Docker images using docker build
- Running containers with port mapping
- Understanding base images
- Image optimization using Alpine Linux
- Reducing image size for efficient deployment

A lightweight optimized image was created using python:3.6-alpine, reducing image size significantly and improving deployment efficiency.

Lab 03

1}

The screenshot displays a Docker Lab interface with a question on the left and a terminal on the right.

QUESTION 1 OF 15

How many `images` are available on this host?

Options: 6, 3, 0, and a visual representation of a bar chart with 6 bars.

Terminal 1

```
~ ➔ docker images
REPOSITORY          TAG         IMAGE ID      CREATED       SIZE
mysql                latest      f6b0ca07d79d 7 months ago 934MB
postgres            latest      a38f9f77ff88 7 months ago 456MB
alpine               latest      706db57fb206 7 months ago 8.32MB
nginx                latest      657fddc1c365 7 months ago 152MB
nginx                alpine     5e7abdd2021 7 months ago 52.7MB
redis                latest      466e5b1da2ef 7 months ago 137MB
ubuntu               latest      97bed23a3497 7 months ago 78.1MB
kodekloud/simple-webapp-mysql latest      129dd9f67367 7 years ago 96.6MB
kodekloud/simple-webapp latest      c6e3cd9aae36 7 years ago 84.8MB
```

||

```
~ ➔ docker images | tail -n +2
mysql                latest      f6b0ca07d79d 7 months ago 934MB
postgres            latest      a38f9f77ff88 7 months ago 456MB
alpine               latest      706db57fb206 7 months ago 8.32MB
nginx                alpine     5e7abdd2021 7 months ago 52.7MB
nginx                latest      657fddc1c365 7 months ago 152MB
redis                latest      466e5b1da2ef 7 months ago 137MB
ubuntu               latest      97bed23a3497 7 months ago 78.1MB
kodekloud/simple-webapp-mysql latest      129dd9f67367 7 years ago 96.6MB
kodekloud/simple-webapp latest      c6e3cd9aae36 7 years ago 84.8MB
```

~ ➔ docker images | tail -n +2 | wc -l
9

~ ➔

2)

Task

Hint

AI Assistant

44:45

QUESTION 2 OF 15

What is the size of the `ubuntu` image?

95MB

5.58MB

78.1MB

1.2GB

64GB

Terminal 1

Stop Lab

```
~ → docker images
REPOSITORY          TAG         IMAGE ID      CREATED       SIZE
mysql                latest      feb0ca07d7d   7 months ago 934MB
postgres            latest      a38f9f77f88   7 months ago 454MB
alpine              latest      786db27fb286   7 months ago 8.32MB
nginx               latest      657fddc1c365   7 months ago 152MB
nginx               alpine     5e7abdd2821   7 months ago 52.7MB
redis               latest      466e5b1da2ef   7 months ago 137MB
ubuntu              latest      97bed23a3497   7 months ago 78.1MB
kodekloud/simple-webapp-mysql latest      129dd9f67367   7 years ago 96.6MB
kodekloud/simple-webapp latest      c6e3cd9aae36   7 years ago 84.8MB

~ → size=$(docker image inspect ubuntu --format="{{.Size}}") && echo $((size/1024)) MB
74 MB

~ → size=$(docker image inspect ubuntu --format="{{.Size}}") && echo $((size/1000/1000)) MB
78 MB

~ →
```

3)

i

53:24

QUESTION 3 OF 15

Preview only — answers can't be changed or retried here.

We just pulled a new image. What is the tag on the newly pulled `NGINX` image?

1.10.1

1.14-alpine

perl

Terminal 1

Stop Lab

```
~ → docker images | grep -i nginx
nginx      alpine     5e7abdd2821   7 months ago 52.7
MB
nginx      latest     657fddc1c365   7 months ago 152M
B
nginx      1.14-alpine 8a2fb25a19f5   7 years ago 16MB

~ →
```

4)

QUESTION 4 OF 15

Preview only — answers can't be changed or retried here.

We just downloaded the code of an application. What is the base image used in the Dockerfile?

Inspect the Dockerfile in the `webapp-color` directory.

python

ubuntu:latest

centos:7

rhel:3.6

python:3.6

Terminal 1

Stop Lab

```
~ → ls -l webapp-color/Dockerfile
-rw-r--r-- 1 root root 113 Jan 29 11:24 webapp-color/Dockerfile

~ → cat webapp-color/Dockerfile
FROM python:3.6

RUN pip install flask

COPY . /opt/

EXPOSE 8080

WORKDIR /opt

ENTRYPOINT ["python", "app.py"]

~ →
```

[illegible]

QUESTION 6 OF 15

Preview only — answers can't be changed or retrieved here.

Based on the given Dockerfile, which instruction defines the command that starts the application when a container is started?

Inspect the Dockerfile in the `webapp-color` directory.

app.exe

pip install flask

docker run app

python app.py

```
~ → ls -l webapp-color/Dockerfile
-rw-r--r-- 1 root root 113 Jan 29 11:24 webapp-color/Dockerfile

~ → cat webapp-color/Dockerfile
FROM python:3.6

RUN pip install flask

COPY . /opt/

EXPOSE 8080

WORKDIR /opt

ENTRYPOINT ["python", "app.py"]

~ →
```

7)

QUESTION 7 OF 15

What `port` is the web application run within the container?

Inspect the Dockerfile in the `webapp-color` directory.

8080

0.0.0.0

80

5000

```

~ → ls -l webapp-color/Dockerfile
-rw-r--r-- 1 root root 113 Jan 29 11:24 webapp-color/Dockerfile

~ → cat webapp-color/Dockerfile

FROM python:3.6

RUN pip install flask

COPY . /opt/

EXPOSE 8080

WORKDIR /opt

ENTRYPOINT ["python", "app.py"]

~ → 

```

8)

QUESTION 8 OF 15

Build a docker image using the Dockerfile and name it `webapp-color`. No tag to be specified.

Check

Next →

Image Name: webapp-color

```

~ → cat webapp-color/Dockerfile

FROM python:3.6

RUN pip install flask

COPY . /opt/

EXPOSE 8080

WORKDIR /opt

ENTRYPOINT ["python", "app.py"]

~ → cd webapp-color

~/webapp-color via  ll
Dockerfile      app.py          requirements.txt templates

~/webapp-color via  Docker build -t webapp-color .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
BuildKit is currently disabled; enable it by removing the DOCKER_BUILDKIT=0
environment-variable.

Sending build context to Docker daemon 8.704kB
Step 1/6 : FROM python:3.6
3.6: Pulling from library/python
0e29546d541c: Pull complete
9b829c73b52b: Pull complete
cb5b7ae36172: Pull complete
6494e4811622: Pull complete
6f9f74896dfa: Pull complete
52b4313c655c: Pull complete

```

9)

QUESTION 9 OF 15

Run an instance of the image `webapp-color` and publish port `8080` on the container to `8282` on the host.

Check

Next →

- Container with image 'webapp-color'
- Container Port: 8080
- Host Port: 8282

```
~/webapp-color via docker run -p 8080:8282 webapp-color
docker: Error response from daemon: driver failed programming external connectivity on endpoint friendly_cray (775115bd1196a2d7f03c9d8c1a6a3d01a774d734c476da486e97755e5d63500f): Error starting userland proxy: listen tcp4 0.0.0.0:8080: bind: address already in use.
ERRO[0000] error waiting for container: context canceled

~/webapp-color via docker run -p 8282:8080 webapp-color
This is a sample web application that displays a colored background.
A color can be specified in two ways.

1. As a command line argument with --color as the argument. Accepts one of red,green,blue,blue2,pink,darkblue
2. As an Environment variable APP_COLOR. Accepts one of red,green,blue,blue2,pink,darkblue

3. If none of the above then a random color is picked from the above list.
Note: Command line argument precedes over environment variable.

No command line argument or environment variable. Picking a Random Color =blue2
* Serving Flask app 'app' (lazy loading)
* Environment: production
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Debug mode: off
* Running on all addresses.
WARNING: This is a development server. Do not use it in a production deployment.
* Running on http://172.12.0.2:8080/ (Press CTRL+C to quit)
```

10)

QUESTION 10 OF 15

Preview only — answers can't be changed or retried here.

Access the application by clicking on the tab named `HOST:8282` above your terminal.

After you are done, you may stop the running container by hitting `CTRL + C` if you wish to.

Ok

Correct Answer

```
~/webapp-color via docker run -p 8080:8282 webapp-color
docker: Error response from daemon: driver failed programming external connectivity on endpoint friendly_cray (775115bd1196a2d7f03c9d8c1a6a3d01a774d734c476da486e97755e5d63500f): Error starting userland proxy: listen tcp4 0.0.0.0:8080: bind: address already in use.
ERRO[0000] error waiting for container: context canceled

~/webapp-color via docker run -p 8282:8080 webapp-color
This is a sample web application that displays a colored background.
A color can be specified in two ways.

1. As a command line argument with --color as the argument. Accepts one of red,green,blue,blue2,pink,darkblue
2. As an Environment variable APP_COLOR. Accepts one of red,green,blue,blue2,pink,darkblue

3. If none of the above then a random color is picked from the above list.
Note: Command line argument precedes over environment variable.

No command line argument or environment variable. Picking a Random Color =blue2
* Serving Flask app 'app' (lazy loading)
* Environment: production
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Debug mode: off
* Running on all addresses.
WARNING: This is a development server. Do not use it in a production deployment.
* Running on http://172.12.0.2:8080/ (Press CTRL+C to quit)
192.168.138.190 - - [24/May/2026 14:51:31] "GET / HTTP/1.1" 200 -

~/webapp-color via
```

11)

QUESTION 11 OF 15

What is the base Operating System used by the `python:3.6` image?

If required, run an instance of the image to figure it out.

rhel

ubuntu

debian

centos

```
~/webapp-color via  docker run -d python:3.6
b1766ceaa7a040c937f423578c37e6fbbc1266adfd625b5e315a4cf96a784ef7

~/webapp-color via  docker run python:3.6 cat /etc/os-release

PRETTY_NAME="Debian GNU/Linux 11 (bullseye)"
NAME="Debian GNU/Linux"
VERSION_ID="11"
VERSION="11 (bullseye)"
VERSION_CODENAME=bullseye
ID=debian
HOME_URL="https://www.debian.org/"
SUPPORT_URL="https://www.debian.org/support"
BUG_REPORT_URL="https://bugs.debian.org/"

~/webapp-color via  
```

12)

QUESTION 12 OF 15

Preview only — answers can't be changed or retried here.

What is the approximate size of the `webapp-color` image?

2GB

920MB

80MB

500MB

Correct Answer

```
~/webapp-color via  ddocker image inspect webapp-color --format '{{.Size}}'
912916134

~/webapp-color via  docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
webapp-color         latest      d1a3961397fc  7 minutes ago  913MB
mysql                latest      f6b0ca07d79d  7 months ago  934MB
postgres             latest      a38f9f77ff88  7 months ago  456MB
alpine               latest      706db57fb206  7 months ago  8.32MB
nginx                latest      657fddcd1c365  7 months ago  152MB
nginx                alpine     5e7abccdd2021  7 months ago  52.7MB
redis                latest      466e5b1da2ef  7 months ago  137MB
ubuntu               latest      97bed23a3497  7 months ago  78.1MB
python               3.6        54260638d07c  4 years ago   902MB
nginx                1.14-alpine 8a2fb25a19f5  7 years ago   16MB
kodekloud/simple-webapp-mysql latest      129dd9f67367  7 years ago   96.6MB
kodekloud/simple-webapp latest      c6e3cd9aae36  7 years ago   84.8MB

~/webapp-color via  bc
bc 1.36.1
Adapted from https://github.com/gavinhoward/bc
Original code (c) 2018 Gavin D. Howard and contributors
912916134/1000/1000
912

```

13)

QUESTION 14 OF 15

Build a new smaller docker image by modifying the same Dockerfile and name it `webapp-color` and tag it `lite`.

Hint: Find a smaller base image for `python:3.6`. Make sure the final image is less than `150MB`.

Check

Next →

✓ Name: `webapp-color:lite`

✓ Image size less than 150MB.

```
~/webapp-color via ccat >> Dockerfile
FROM python:3.6-alpine
RUN pip install flask
COPY . /opt/
EXPOSE 8080
WORKDIR /opt
ENTRYPOINT ["python", "app.py"]

~/webapp-color via cdocker build -t webapp-color:lite .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
BuildKit is currently disabled; enable it by removing the DOCKER_BUILDKIT=0
environment-variable.

Sending build context to Docker daemon 8.704kB
Step 1/12 : FROM python:3.6
--> 54260638d07c
Step 2/12 : RUN pip install flask
--> Using cache
--> c57d6334392c
Step 3/12 : COPY . /opt/
--> bc9ef5e91a19
Step 4/12 : EXPOSE 8080
--> Running in 4293600bca80
--> Removed intermediate container 4293600bca80
--> 623cf97a9467
Step 5/12 : WORKDIR /opt
--> Running in ad51dd8982ec
--> Removed intermediate container ad51dd8982ec
--> 54bf082e80dd
Step 6/12 : ENTRYPOINT ["python", "app.py"]
--> Running in 51c1255681da
--> Removed intermediate container 51c1255681da
--> 806588f5110f
Step 7/12 : FROM python:3.6-alpine
3.6-alpine: Pulling from library/python
```

15)

QUESTION 15 OF 15

Run an instance of the new image `webapp-color:lite` and publish port `8080` on the container to `8383` on the host.

Check

Next →

✓ Container with image 'webapp-color:lite'

✓ Container to publish port 8080 to 8383

```
~/webapp-color via ddocker run -p 8383:8080 webapp-color:lite
This is a sample web application that displays a colored background.
A color can be specified in two ways.

1. As a command line argument with --color as the argument. Accepts one of red,green,blue,
blue2,pink,darkblue
2. As an Environment variable APP_COLOR. Accepts one of red,green,blue,blue2,pink,darkblue

3. If none of the above then a random color is picked from the above list.
Note: Command line argument precedes over environment variable.

No command line argument or environment variable. Picking a Random Color =red
* Serving Flask app 'app' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on all addresses.
  WARNING: This is a development server. Do not use it in a production deployment.
* Running on http://172.12.0.2:8080/ (Press CTRL+C to quit)
```

Docker Lab 4 Short Note

Docker Lab 4 focused on environment variables, container inspection, and MySQL container deployment.

In this lab, environment variables were passed into containers using the `-e` option. Container details and environment configurations were inspected using `docker inspect`.

The lab also covered:

- Using environment variables inside containers
- Running named containers
- Port publishing and mapping
- Inspecting container configurations
- Deploying a MySQL database container securely

A MySQL container was deployed with a root password using environment variables, helping understand real-world containerized database deployment practices.

Based on uploaded Docker lab notes:

Task

Hint

AI Assistant

52:39

QUESTION 1 OF 4

Preview only — answers can't be changed or retried here.

Inspect the environment variables set on the running container and identify the value set to the `APP_COLOR` variable.

yellow

green

pink

blue

Correct Answer

Terminal 1

HOST:38282

Stop Lab

```
~ → docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS        NAMES
1b534a3ca399   kodekloud/simple-webapp "python app.py"         6 minutes ago   Up 6 minutes   8080/tcp     affectionate_m
claren

~ → docker inspect --format="{{.Config.Env}}" 1b
[APP_COLOR=pink PATH=/usr/local/bin:/usr/local/sbin:/usr/sbin:/usr/bin:/bin LANG=C.UTF-8 GPG_K
EY=0D960F4D4110E5C3F8FB17F2D347E6A6A65421D PYTHON_VERSION=3.6.6 PYTHON_PIP_VERSION=18.1]

~ → docker inspect 1b | grep APP
      "APP_COLOR=pink",

~ →
```

Task

Hint

AI Assistant

49:01

QUESTION 2 OF 4

Run a container named `blue-app` using image `kodekloud/simple-webapp` and set the environment variable `APP_COLOR` to `blue`. Make the application available on port `38282` on the host. The application listens on port `8080`.

Check

Next →

✓ Name: blue-app

✓ Image: kodekloud/simple-webapp

✓ ENV Variable: APP_COLOR=blue

Terminal 1

HOST:38282

Stop Lab

```
~ → docker run --name blue-app -p 38282:8080 -e APP_COLOR=blue kodekloud/simple-webapp
This is a sample web application that displays a colored background.
A color can be specified in two ways.

1. As a command line argument with --color as the argument. Accepts one of red,green,blue,blue2,pink,darkblue
2. As an Environment variable APP_COLOR. Accepts one of red,green,blue,blue2,pink,darkblue
3. If none of the above then a random color is picked from the above list.
Note: Command line argument precedes over environment variable.

No Command line argument, Color from environment variable =blue
* Serving Flask app "app" (lazy loading)
* Environment: production
WARNING: Do not use the development server in a production environment.
Use a production WSGI server instead.
* Debug mode: off
* Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)

~
```

Task

Hint

AI Assistant

47:24

QUESTION 3 OF 4

Preview only — answers can't be changed or retried here.

1 View the application by clicking the link `HOST:38282` on the right side of your terminal bar and ensure it has the right colour.

OK

Terminal 1

HOST:38282

Stop Lab

```
~ → docker run --name blue-app -p 38282:8080 -e APP_COLOR=blue kodekloud/simple-webapp
This is a sample web application that displays a colored background.
A color can be specified in two ways.

1. As a command line argument with --color as the argument. Accepts one of red,green,blue,blue2,pink,darkblue
2. As an Environment variable APP_COLOR. Accepts one of red,green,blue,blue2,pink,darkblue
3. If none of the above then a random color is picked from the above list.
Note: Command line argument precedes over environment variable.

No Command line argument, Color from environment variable =blue
* Serving Flask app "app" (lazy loading)
* Environment: production
WARNING: Do not use the development server in a production environment.
Use a production WSGI server instead.
* Debug mode: off
* Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
10.244.132.130 - - [24/May/2026 14:17:22] "GET / HTTP/1.1" 200 -

~
```

Labs - Environment Var... x Hello from Flask x 0.0.0.0 x What is the size of the x (3) Cryptography Full C x Inspect the environm... x Ask Gemini x

38282-port-pd2r4jwmxweqjua0Labs.kodekloud.com

Apps x GitHub x ChatGPT x Swayam Central x TudeDude x Gmail x YouTube x Home Page | InfoTQ x LinkedIn x Maps x RemixDE x Smart Phone contro... x Interface a Solenoid... x

Hello from 4fbdc3ce74bd!

TaskHintAI Assistant

43:46

Terminal 1

HOST:38282Stop Lab

QUESTION 4 OF 4

Deploy a `mysql` database using the `mysql` image and name it `mysql-db`.

Set the database password to use `db_pass123`. Lookup the `mysql` image on Docker Hub and identify the correct environment variable to use for setting the root password.

Check

Next →

Name: mysql-db

Image: mysql

Env: MYSQL_ROOT_PASSWORD=db_pass123

```
~ → docker run --name mysql-db -e MYSQL_ROOT_PASSWORD=db_pass123 mysql
2026-05-24 14:21:13:00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 9.5.0-1.el9 started.
2026-05-24 14:21:14:00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
2026-05-24 14:21:14:00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 9.5.0-1.el9 started.
2026-05-24 14:21:15:00:00 [Note] [Entrypoint]: Initializing database files
2026-05-24:14:21:15:243045Z 0 [System] [MY-015017] [Server] MySQL Server Initialization - start.
2026-05-24:14:21:15:249412Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 9.5.0) initializing of server
in progress as process 230
2026-05-24:14:21:15:304119Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-05-24:14:21:16:023268Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-05-24:14:21:18:256934Z 6 [Warning] [MY-010453] [Server] root@localhost is created with an empty password ! Ple
ase consider switching off the --initialize-insecure option.
2026-05-24:14:21:21:206388Z 0 [System] [MY-015018] [Server] MySQL Server Initialization - end.
2026-05-24 14:21:21:400:00 [Note] [Entrypoint]: Database files initialized
2026-05-24 14:21:21:400:00 [Note] [Entrypoint]: Starting temporary server
2026-05-24:14:21:21:256790Z 0 [System] [MY-015015] [Server] MySQL Server - start.
2026-05-24:14:21:21:654212Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 9.5.0) starting as process 277
2026-05-24:14:21:21:654236Z 0 [System] [MY-015590] [Server] MySQL Server has access to 16 logical CPUs.
2026-05-24:14:21:21:654255Z 0 [System] [MY-015590] [Server] MySQL Server has access to 66486943744 bytes of physica
l memory.
2026-05-24:14:21:21:739428Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-05-24:14:21:22:411148Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-05-24:14:21:22:806258Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
2026-05-24:14:21:22:806275Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted
connections are now supported for this channel.
```